Honov5 RCを触ってみる

～ v4 と並べて、何が壊れるのか確かめた ～

田中博悠 / tanahiro2010

田中博悠 / tanahiro2010

- 三田学園高等学校 1年
- Hono Conference 2026 コアスタッフ
- 個人開発でずっと Hono を使ってる

Hono、好きです



Hono v5 RC、出ましたね

v5 RC

- 10月8日 → `5.0.0-rc.0`
- 10月10日 → `5.0.0-rc.1`

honojs/hono releases

SH

```
npm i hono@next
```

もう触れます

ح

あなたのアプリ、

v5 にしたら壊れますか？

僕のは壊れました

方針

Migration Guide の翻訳はしない

やったこと

- v4 と v5 の環境を**別々に**作る
- **同じコードを両方で動かす**
- 差分だけを見る

「ドキュメントにこう書いてある」

↓

「実際にこうなった」

環境

v4	`hono@4.13.13`
v5	`hono@5.0.0-rc.1`
Node.js	v24.22.0
TypeScript	7.0.2

Apple M4 / 2026-10-18 時点

水を飲む

型

`c.req.json()`

```ts
app.post('/users', async (c) => {
  const body = await c.req.json()

  return c.json({ name: body.name.toUpperCase() })
})
```

- v4 → `any`
- v5 → `unknown`

v4 では、このコードが通る

v5

```
error TS18046: 'body' is of type 'unknown'.
```

ちゃんと止まった

もうひとつ気になってたのが

`c.req.query()`

```ts
const { name } = c.req.query()
const upper: string = name.toUpperCase()
```

- v4 → `Record<string, string>`
- v5 → `Record<string, string | undefined>`

これ、分割代入してるコードが

全部赤くなるのでは？

v5

```
error TS18048: 'name' is possibly 'undefined'.
error TS2322: Type 'string[] | undefined' is not
  assignable to type 'string[]'.
```

全部赤くなった

でも、ここで面白いことに気づいた

実行時の値を比べてみる

`GET /q?a=1&a=2`

	v4	v5
`query()`	`{"a":"1"}`	`{"a":"1"}`
`queries()`	`{"a":["1","2"]}`	`{"a":["1","2"]}`

完全に同じ

わかったこと

- v5 は挙動を**変えていない**
- v4 でも `undefined` は返っていた
- 型が嘘をついていただけ

赤くなった箇所は、

v4 の時点でバグり得た箇所

水を飲む

エラー

ここが v5 で一番面白い

v4 の `onError()`

```ts
app.onError((err, c) => {
  console.error(err.message)
  return c.json({ error: 'Internal Server Error' }, 500)
})
```

- アプリ全体に1つだけ
- シグネチャは `(err, c)`

v5 の `onError()`

```ts
app.onError('/api/*', async (c, next) => {
  console.error(c.error?.message)
  c.header('X-Error-Handler', 'api')
  await next()
})

app.onError((c) => c.json({ error: 'Error' }, 500))
```

- `(c, next)` → 普通のミドルウェア
- エラーは `c.error` から読む
- **パスで絞れる**

設計意図

> エラー処理を Hono の**ルーティングシステムの上に**
> モデル化する

- 内部では `@ERROR` / `@NOT_FOUND` という

予約メソッドで Router に登録されている

honojs/hono#5541

本当にパスで絞れるのか？

動かしてみた

リクエスト	ヘッダ	status
`/api/error`	`api`	500
`/other/error`	なし	500

効いた

ちなみに v4 で同じことを書くと

v4

```
this.errorHandler is not a function
```

- パス文字列をハンドラとして保存してしまう
- **登録時ではなくリクエスト時に落ちる**

ここからが本番

`onError()` を 2回登録したら

どうなる？

実験

```ts
app.onError((c) => { order.push('first');  ... })
app.onError((c) => { order.push('second'); ... })
```

結果

	実行されたハンドラ
v4	`["second"]`
v5	`["first", "second"]`

v4 は上書き

v5 は合成

敗因になりそうなところ

- v4 → 「あとから登録して潰す」が書けた
- v5 → **両方走る**
- しかも…

型エラーにならない

シグネチャさえ直せば `tsc` は通る

じゃあ v4 の書き方のまま

動かしたらどうなる？

型チェックは止めてくれる

```
error TS2339: Property 'message' does not exist
  on type 'Context<BlankEnv, string, any>'.
error TS2339: Property 'json' does not exist
  on type 'Next'.
```

- 第1引数が `Context`、第2引数が `Next`

問題は、これを握りつぶしたとき

実行すると

```
Context is not finalized.
Did you forget to return a Response object
or `await next()`?
```

原因を指していない

何が起きてるか

- `c` のつもりで触ってるものが `next` 関数
- `c.text()` が無い → 例外
- → Context が確定しないまま終わる

型エラーを消すと、デバッグが遠回りになる

非 Error を `throw` したら

	v4	v5
`throw 'oops'`	外に漏れる	`message` / `cause` に `'oops'`
`throw { code: 42 }`	外に漏れる	`message` は **空**

- オブジェクトは `cause` にしか入らない
- → `c.error.message` だけログに出すと何も分からない

水を飲む

地雷

型でも捕まらないやつ

`c.json(undefined)`

	v4	v5
`c.json(undefined)`	200 / 空ボディ	500
`c.json(() => 1)`	200 / 空	500

```
TypeError: Value is not JSON serializable
```

型エラーにならない

どこで踏むか

```ts
const user = await db.query.users.findFirst({ ... })
return c.json(user) // 見つからなければ undefined
```

- v4 → 404 のつもりが 200 で空ボディ
- v5 → **500**

つまり

v5 で 500 が出る箇所は

v4 の時点でバグっていた

箇所

水を飲む

ルーター

ワイルドカードが統一された

v4 は Router ごとにバラバラだった

v4

ルート	パス	Smart	RegExp	Trie	Pattern
`/x*/y`	`/x/y`	❌	❌	❌	❌
`/x*/y`	`/xz/y`	✅	✅	❌	❌
`/x*y`	`/xay`	✅	✅	❌	❌
`/wild/*/card`	`/wild//card`	❌	❌	✅	❌

6ケース中3ケースで割れている

v5

ルート	パス	Smart	RegExp	Trie	Pattern
`/x*/y`	`/x/y`	✅	✅	✅	✅
`/x*/y`	`/xz/y`	✅	✅	✅	✅
`/x*y`	`/xay`	❌	❌	❌	❌
`/wild/*/card`	`/wild//card`	❌	❌	❌	❌

4つとも一致

ここが一番おいしい

競合するルートを書いてみる

```ts
app.get('/x*/y', (c) => c.text('a'))
app.get('/x/*',  (c) => c.text('b'))
```

v5 の RegExpRouter

```
UnsupportedPathError: /x/*
```

- 両方を同時に登録できない

じゃあ SmartRouter はどうなる？

選ばれた Router

登録したルート	Router
`/x/*` だけ	`SmartRouter + RegExpRouter`
`/x*/y` + `/x/*`	**`SmartRouter + TrieRouter`**

ルートを1行足しただけで

Routerが入れ替わった

性能も変わるのでは？

測った

`GET /users/:id` × 50,000回 × 12ラウンド

	中央値	最小
RegExpRouter	138.68 ms	132.35 ms
Trie フォールバック	148.57 ms	146.25 ms
差	+7.1%	+10.5%

遅くなった

でも問題はそこじゃない

- 7〜10% は正直どうでもいい
- 問題は**無言で切り替わる**こと
- ログにも型にも出ない

しかも

マッチするハンドラも変わる

同じ競合ルートに `/x/y` を投げると

v4	v5
`/x/*` の方	`/x*/y` の方

水を飲む

性能

v5、速くなったの？

測った

`GET /users/:id` × 100,000回 × 10ラウンド

	中央値	最小
v4	270.53 ms	258.31 ms
v5	266.50 ms	255.59 ms

`app.fetch()` のインプロセス計測。実サーバーの性能ではない

誤差

分布が重なっている

- v4 の最小 **258.31 ms**
- v5 の中央値 **266.50 ms**
- → v4 の方が速いラウンドがある

ここで、恥ずかしい話を

敗因

- 最初 **5ラウンド**で測った → 差 **2.9%**
- **10ラウンド**に増やした → 差 **1.5%**
- 同じコード、同じマシン、同じ日

ラウンド数を変えただけで

結論が倍違った

わかったこと

- マイクロベンチで数%を語るなら

ラウンド数と分布を出す

- Router の方も 3ラウンド → 12ラウンドで

3.3% → 7.1% に変わった

コアの性能は据え置き

v5 の変更は、性能のためじゃない

水を飲む

サイズ

「アダプタ分離で軽くなった」

本当？

インストールサイズ

	展開後	ファイル数
v4	1.23 MB	574
v5	**0.66 MB**	**310**

npm registry の unpackedSize

-46%

3つの測り方が一致した

- npm registry → **-46%**
- 手元の `du -sh`（2.8M → 1.5M）→ **-46%**
- yusukebeさんの資料（1.29MB → 0.70MB）→ **-46%**

ファイル数 574 → 310 は完全一致

じゃあバンドルは？

バンドルサイズ

	v4	v5	差
コアのみ	18,528 B	18,830 B	+302 B
アダプタ込み	22,760 B	22,923 B	+163 B

増えた

考えてみれば当たり前

- 減ったのは CJS ビルドとアダプタの分離
- CJS はそもそもバンドルに入らない
- `@hono/bun` を入れれば同じコードが入る

「軽くなった」は

インストールサイズの話

水を飲む

設計

ここまで全部、同じ方向を向いてる

v5 = AIのためのHono

> 抽象化されていて、選択肢が少ない

Yusuke Wada / Hono Conference 2026

なぜ型を厳しくするのか

- 人間は「ここは `any` だけど大丈夫」と空気を読む
- **LLM は読まない**
- 型が嘘をつけば、嘘のまま出てくる

それが一番出てるのが

`defineHandler()`

```ts
app.get('/ping', defineHandler(() => ({ pong: true })))
```

```
ping -> 200 {"pong":true}
```

- `c.json()` を呼んでいない
- **返した値が Response になる**

検証も1か所で書ける

```ts
app.post('/users', defineHandler({ json: NameSchema })(
  (c, input) => ({ name: input.json.name.toUpperCase() })
))
```

```
400 {"error":"Validation failed","issues":[...]}
```

- Standard Schema → Zod / Valibot ...

ハマった

- 普通の検証関数で `throw` すると

400 ではなく **500** になる

- 400 にしてくれるのは Standard Schema のルートだけ

型定義のコメント

> A JSON Response by `c.json()` is not accepted,
> because it bypasses the response validation.

- `c.json()` を返すことを**型で禁止**している

やり方が複数あって、どれでも動く

↓

正しいやり方が1つで、他は型で止まる

水を飲む

まとめ

移行チェックリスト

①まず `tsc --strict` を通す

- `c.req.json()` の直アクセス
- `c.req.query()` の分割代入
- `onError((err, c) => ...)`

→ 機械的に潰せる

移行チェックリスト

②型では捕まらない3つ

- `c.json(undefined)` → 500
- `onError()` の合成化 → 両方走る
- ワイルドカードのマッチ変更

→ **ここが本番**

移行チェックリスト

③import を書き換える

- `hono/<runtime>` → `@hono/<runtime>`
- `hono/cloudflare-pages` → 代替なし
- `app.mount()` → `hono/mount`
- `app.fire()` → `@hono/service-worker`

まとめ

- 型 → 嘘をつかなくなった。実行時の値は同じ
- エラー → 上書きから**合成**へ
- 地雷 → `c.json(undefined)` が 500
- ルーター → 統一された。でも無言で入れ替わる
- 性能 → 据え置き。ラウンド数を書こう
- サイズ → install は -46%、バンドルは微増

v5 で壊れるのは

v4 が緩かったところ

壊れたんじゃなくて、

隠れてたバグが出てきただけ

本番投入は RC なので待ちます

でも `hono@next` で1回ビルドするのは

今やる価値がある

型エラーの量を見れば、

移行コストがその場で分かる

ありがとうございました

One more thing

正式リリースまでの予想

ひっくり返るとしたら

`defineHandler()` だと思う

- 唯一 experimental と明記されている
- 検証関数で 500 になる角が残っている
- `onError()` とワイルドカードはこのまま出る

一番詰まるのは

`hono/cloudflare-pages` の削除

- 他は書き換えれば済む
- これだけ**代替がない**
- 型エラーの数なら `c.req.query()` が圧倒的
- でも時間を食うのは**数じゃなくて1件**

外れるかもしれません

正式版が出たら、答え合わせします

```sh
npm i hono@next
```

hono.dev/docs/guides/migrating-to-v5